



DealHunter - Report di progetto

Francesco Coppola - 0512119837

February 2026

Indice

1	Scenario e Problema Analizzato	3
1.1	Contesto Applicativo	3
1.2	Problema Affrontato	3
1.3	Task	3
2	Dataset Utilizzato e Caratteristiche	4
2.1	Fonte e Dimensioni	4
2.2	Descrizione Feature Numeriche	4
2.3	Descrizione Feature Categoricalhe	5
2.4	Feature Ingegnerizzate	5
3	Issue Analizzate e Soluzioni Proposte	5
3.1	Issue 1: Zeri Fittizi (Missing Values Mascherati)	5
3.2	Issue 2: Identificazione Outlier	7
3.3	Issue 3: Distribuzione Sbilanciata dei Prezzi	8
3.4	Issue 4: Feature Categoricalhe ad Alta Cardinalità	9
3.5	Issue 6: Data Drift	10
4	Analisi delle Prestazioni	10
4.1	Training Split	10

4.2	Modelli Confrontati	11
4.3	Risultati Numerici Post Training	11
4.4	Correlazione fra le feature	14
4.5	Motivazione Scelta finale: Logistic Regression	15
5	Considerazioni finali	16
5.1	Risultati Raggiunti	16
5.2	Limitazioni Riconosciute del modello	16
5.3	Conclusione	16

Elenco delle figure

1	Missing Values	5
2	Identificazione Outlier (Tablet)	7
3	Distribuzione dei prezzi	8
4	Distribuzione classi target dopo il cut	9
5	Confronto Modelli	11
6	Confronto Metriche Modelli	12
7	Confronto Matrici Di Confusione	13
8	Heatmap di correlazione	14
9	Feature Importance	15

Elenco delle tabelle

1	Definizione del problema di classificazione.	3
2	Statistiche generali del dataset.	4
3	Dettaglio delle feature numeriche.	4
4	Dettaglio delle feature categoriche.	5
5	Feature derivate (Feature Engineering).	5
6	Issue 1: Zeri fittizi.	6
7	Modelli confrontati.	11
8	Motivazione della scelta finale.	15
9	Limitazioni Riconosciute.	16

1 Scenario e Problema Analizzato

1.1 Contesto Applicativo

Il mercato degli smartphone usati rappresenta un settore in costante crescita, trainato dalla sostenibilità ambientale e dall'accessibilità economica. Tuttavia, la determinazione del "giusto prezzo" per un dispositivo usato rimane una sfida complessa, influenzata da molteplici fattori: specifiche tecniche, brand e età del dispositivo.

1.2 Problema Affrontato

Il progetto DealHunter affronta il problema della classificazione del prezzo di smartphone usati in categorie predefinite. L'obiettivo è sviluppare un sistema di Machine Learning in grado di:

- Analizzare le specifiche tecniche di un dispositivo (RAM, fotocamera, batteria, storage, ecc.)
- Predire la fascia di prezzo a cui appartiene il dispositivo
- Fornire una stima interpretabile per utenti finali

1.3 Task

Il problema è stato formulato come un task di classificazione multiclasse supervisionata:

Aspetto	Descrizione
Tipo	Classificazione Multiclasse
Input	Vettore di feature numeriche e categoriche
Output	Classe target: Budget, Mid-Range, High-End, Premium

Tabella 1: Definizione del problema di classificazione.

2 Dataset Utilizzato e Caratteristiche

2.1 Fonte e Dimensioni

Attributo	Valore
Nome	used_device_dataset
Fonte	Kaggle
Righe Originali	3454
Righe Post-Processing	3183
Colonne Originali	15
Feature Finali	47 (post-encoding)

Tabella 2: Statistiche generali del dataset.

2.2 Descrizione Feature Numeriche

Feature	Descrizione	Unità
screen_size	Diagonale Schermo	cm
rear_camera_mp	Risoluzione fotocamera posteriore	MP
front_camera_mp	Risoluzione fotocamera anteriore	MP
internal_memory	Storage interno	GB
ram	Memoria RAM	GB
battery	Capacità batteria	mAh
weight	Peso dispositivo	g
release_year	Anno di rilascio	anno
days_used	Giorni di utilizzo	giorni
normalized_used_price	Prezzo normalizzato	valuta

Tabella 3: Dettaglio delle feature numeriche.

2.3 Descrizione Feature Categoriche

Feature	Descrizione	Valori / Note
device_brand	Marca del dispositivo	42 brand distinti
os	Sistema operativo	Android, iOS, Windows, Others
4g	Supporto rete 4G	yes/no
5g	Supporto rete 5G	yes/no

Tabella 4: Dettaglio delle feature categoriche.

2.4 Feature Ingegnerizzate

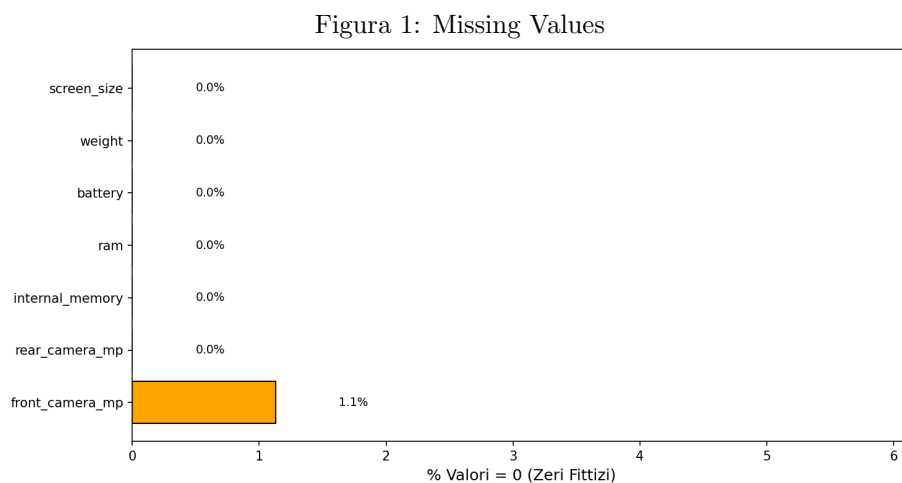
Feature Nuova	Logica di Creazione	Scopo
price_category	pd.qcut(price,4)	Target bilanciato con quartili
model_age	2020 – release_year	Età dispositivo

Tabella 5: Feature derivate (Feature Engineering).

3 Issue Analizzate e Soluzioni Proposte

3.1 Issue 1: Zeri Fittizi (Missing Values Mascherati)

Problema Identificato Nel dataset originale, i valori mancanti non erano rappresentati come NaN o celle vuote, ma come zeri.



Questo è semanticamente scorretto per molte feature:

Feature	Valore	Problema
ram	0 GB	Impossibile, ogni dispositivo ha una ram
weight	0 g	Impossibile, ogni dispositivo ha un peso
battery	0 mAh	Impossibile, ogni dispositivo ha una batteria
rear_camera_mp	0 MP	Molto raro, solo su device antichi

Tabella 6: Issue 1: Zeri fittizi.

Motivazione Evidentemente il dataset originale prevedeva la sostituzione di tutti i missing values numerici con degli 0.

Soluzione Implementata Strategia a due livelli:

- Conversione degli zero in NaN
- Imputazione con mediana locale (release_year, brand)
- Fallback con mediana globale se il gruppo è troppo piccolo

Motivazione delle scelte L'analisi esplorativa ha evidenziato la presenza di valori pari a 0 in variabili dove tale valore è semanticamente impossibile, indicando l'uso dello zero come placeholder per dati mancanti. Si è scelto di convertire esplicitamente tali zeri in NaN per due motivi:

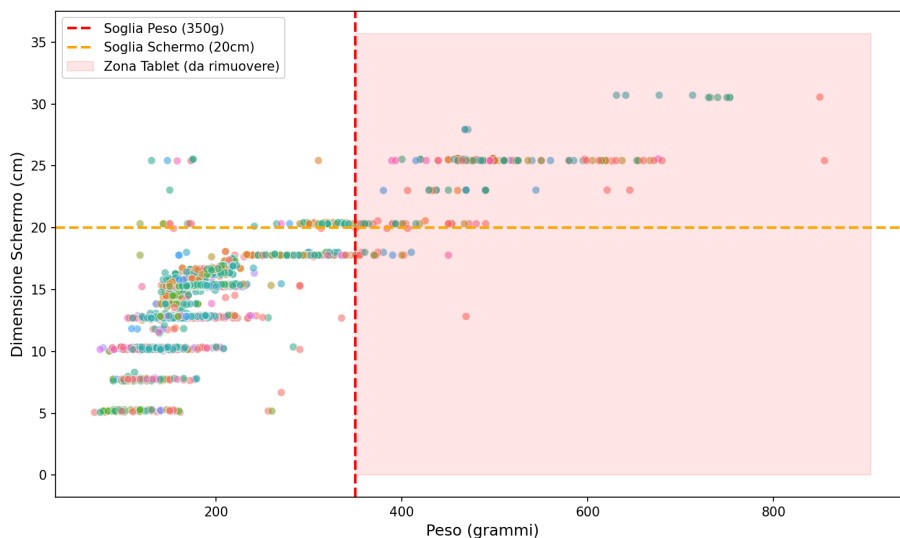
- **Coerenza semantica:** Distinguere l'assenza di informazione dal valore numerico zero.
- **Compatibilità tecnica:** Permettere l'utilizzo degli algoritmi di imputazione standard (che ignorano o gestiscono specificamente i NaN durante il calcolo delle statistiche).

Una **Mediana locale** è una soluzione domain-driven, di fatto consideriamo brand e anno come criterio di selezione del gruppo per dare un senso più logico alla nostra imputazione. **Es.** Un Samsung del 2020 ha un missing value sulla colonna rear_camera_mp? ha molto più senso imputare questo dato sulla base degli altri Samsung del 2020. è comunque possibile che vi siano gruppi troppo piccoli per l'imputazione, è stata quindi implementata una soluzione **FallBack** con la mediana globale della colonna.

3.2 Issue 2: Identificazione Outlier

Problema Identificato Il dataset di partenza conteneva dispositivi con peso maggiore di 300g e schermo maggiore di 20cm, questi sono Tablet e non smartphone.

Figura 2: Identificazione Outlier (Tablet)



Sono evidenti circa 270 dispositivi identificati come Tablet, circa l'8% del dataset.

Soluzione Implementata Rimozione di tutti i dispositivi tablet identificati dal parametro peso maggiore di 300g & schermo maggiore di 20cm.

Motivazione della scelta

- Banalmente il task è classificazione di smartphone, non tablet.
- Includere tablet distorcerebbe le distribuzioni delle feature.
- Le soglie di 300g e 20cm sono basate sulla conoscenza del dominio, gli smartphone più pesanti (rugged) superano molto raramente i 230g e i display maggiore di 20cm sono particolarmente difficili (solo i foldable sono smartphone con display simili ma non sono considerati nel dataset che ha un data drift del 2020, anno in cui i foldable erano un lontano sogno)

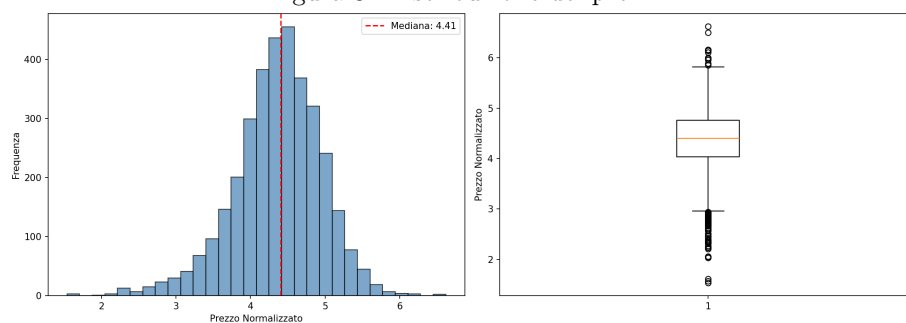
3.3 Issue 3: Distribuzione Sbilanciata dei Prezzi

Problema Identificato la colonna target originale (`normalized_used_price`) aveva una distribuzione continua e sbilanciata, aveva un picco verso i prezzi bassi e una coda lunga sui prezzi alti. Per la classificazione, va convertita in categorie discrete. Ci sono due possibili approcci: **cut** e **qcut**. L'approccio **cut** è stato scartato per alcuni motivi:

- La distribuzione dei prezzi è sbilanciata (skewed)
- Ci sono molti dispositivi economici, pochi super costosi

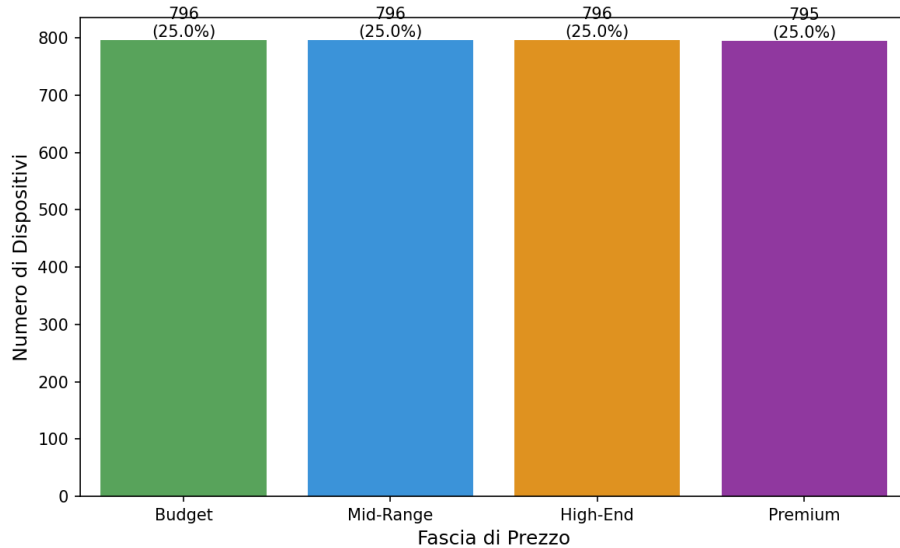
Se usassimo **cut** (uguale ampiezza) avremo un bias, specificamente un class imbalance bias. Con un bias del genere il modello potrebbe anche avere un alta accuracy ma dettata dal predire sempre la classe di maggioranza. Con **qcut** forziamo il bilanciamento del dataset, così il modello deve distinguere realmente tra le classi. In questo modo ogni classe ha lo stesso numero di esempi e quindi il modello deve imparare a distinguere, non può "barare".

Figura 3: Distribuzione dei prezzi



Soluzione implementata La metodologia di taglio **qcut** per il taglio bilanciato in quartili del dataset

Figura 4: Distribuzione classi target dopo il cut



La soluzione del taglio in quartili come si può vedere dal grafico bilancia perfettamente le 4 classi target (25% ciascuna).

3.4 Issue 4: Feature Categorie ad Alta Cardinalità

Problema identificato Durante l'analisi esplorativa, è emerso che il dataset conteneva oltre 30 produttori diversi (high cardinality), molti dei quali con pochissime osservazioni (es. Alcatel, Celkon). Mantenere una colonna dedicata per ogni brand minore avrebbe comportato due problemi:

- **Sparsità dei dati:** Creazione di feature con varianza quasi nulla, inutili per il modello.
- **Incoerenza in Produzione:** L'interfaccia utente è progettata per gestire i principali brand di mercato; brand di nicchia inseriti dall'utente verrebbero logicamente mappati come "Altro".

A seguito delle analisi delle possibili soluzioni, si è riscontrata inadeguatezza del Label Encoding (assegnazione di valori interi sequenziali, es. *Apple* = 1, *Samsung* = 2, ...). L'utilizzo di interi introdurrebbe arbitrariamente una relazione d'ordine ($Brand_A > Brand_B$) e una metrica di distanza fittizia, portando il modello a interpretare erroneamente che un brand con codice numerico più alto abbia un "valore" intrinseco maggiore rispetto a uno con codice più basso, biasando la predizione del prezzo.

Soluzione Implementata Per risolvere ciò, è stata implementata una logica di raggruppamento preventivo: prima del One-Hot Encoding, tutti i brand non presenti nella lista dei "Top Market Share" (definiti nella configurazione del progetto) sono stati ricodificati sotto l'etichetta cumulativa "Others". Successivamente, è stato applicato il One-Hot Encoding, generando un numero fisso e gestibile di feature binarie, garantendo la perfetta coerenza tra le feature di addestramento e gli input dell'applicazione web.

Motivazione della scelta Nel dominio degli smartphone, il brand è un forte predittore del prezzo (es. il premio di prezzo associato ad Apple o Sony rispetto a brand low-cost). Creare colonne separate permette al modello di assegnare un peso (coefficiente) indipendente e specifico per ogni produttore. Il modello può così apprendere che la presenza del valore 1 nella colonna brand_Apple ha un impatto positivo sulla fascia di prezzo, mentre brand_Wiko ne ha uno negativo o neutro, senza dover mediare tra loro.

3.5 Issue 6: Data Drift

Problema Identificato il dataset contiene dispositivi fino al 2020, per i più recenti la feature ingegnerizzata model_age diventa negativa o zero.

Soluzione Implementata I dispositivi con anno maggiore al 2020 vengono trattati con model_age = 0.

Limitazione riconosciuta Tale soluzione è parziale, specifiche di dispositivi 2021+ potrebbero portare il modello a sottostimarne i prezzi. Il modello potrebbe inoltre trattare dispositivi TOP del 2020 come TOP di gamma attuali.

Soluzione Futura Il retraining del modello su dataset aggiornati all'anno corrente.

4 Analisi delle Prestazioni

4.1 Training Split

È stato scelto di dividere il dataset in 80% Training set e 20% Test set, inoltre è stato scelto di stratificare il dataset per ottenere una distribuzione equa delle classi nel training e test set.

4.2 Modelli Confrontati

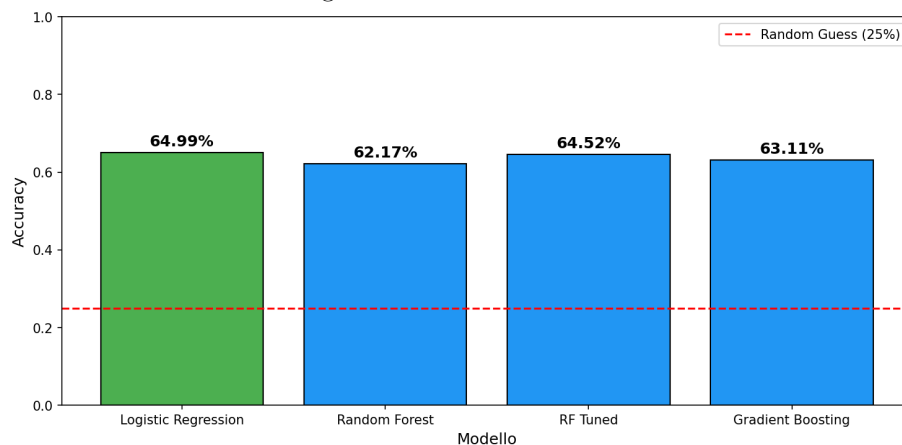
Modello	Descrizione	IperParametri
LogisticRegression	Classificazione Lineare	max_iter=1000
Random Forest	Ensemble di alberi decisionali	n_estimators=100
Random Forest Tuned	Random forest ottimizzato con GridSearchCV	n_estimators=[100, 200], max_depth=[10, 20, None]
Gradient Boosting	Boosting Sequenziale	n_estimators=100

Tabella 7: Modelli confrontati.

Nota su RF Tuned Il modello Random Forest ha le prestazioni più scadenti, per ottimizzarlo e cercare di ottenere un guadagno migliore si è pensato di testare anche la versione Tuned con GridSearchCV, di fatto il Random Forest ha iperparametri di 100 alberi senza limite di profondità mentre RF Tuned valuta tutte le 36 combinazioni e sceglie la migliore.

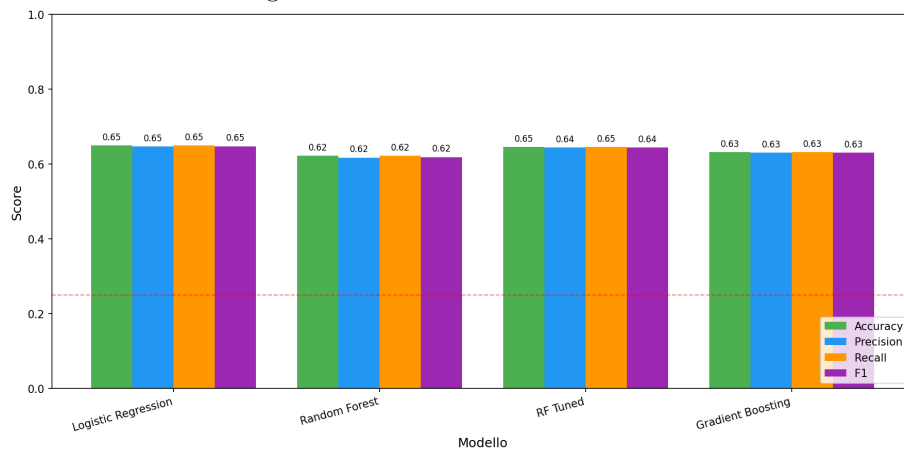
4.3 Risultati Numerici Post Training

Figura 5: Confronto Modelli



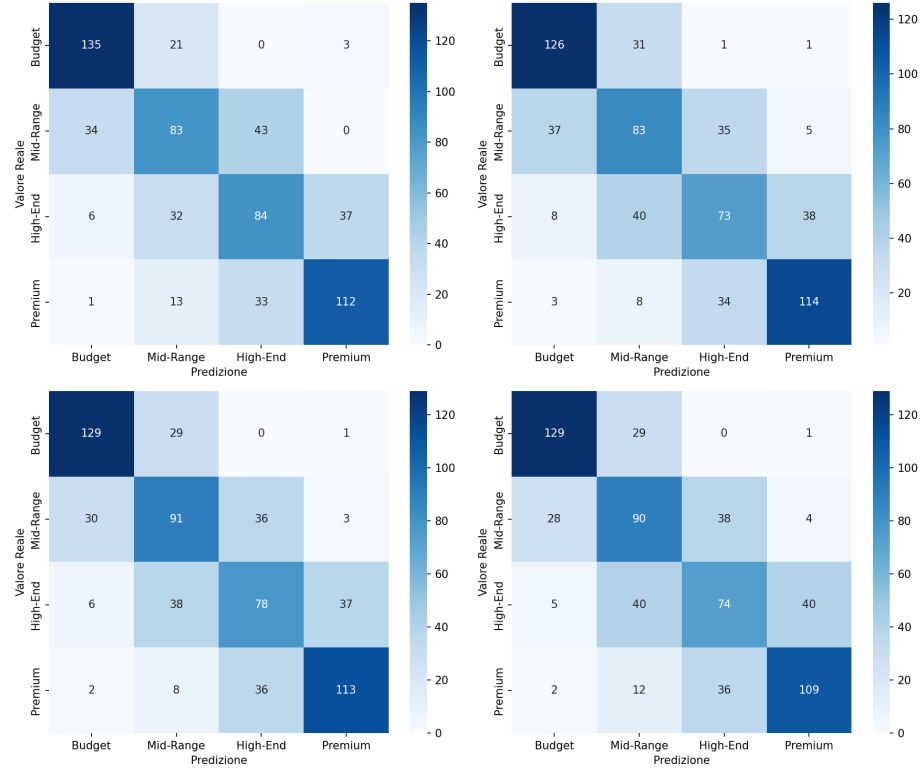
L'Accuracy dei modelli dimostra ci porta a selezionare subito il Logistic Regression, ma conviene approfondire anche le altre metriche di valutazione (F1-Score, Recall e precision) per definire quale modello ha le prestazioni più alte.

Figura 6: Confronto Metriche Modelli



Interpretazione Logistic Regression domina su tutte le metriche. RF Tuned non migliora significativamente RF base (overfitting o saturazione). Le metriche sono consistenti (no trade-off forte precision/recall).

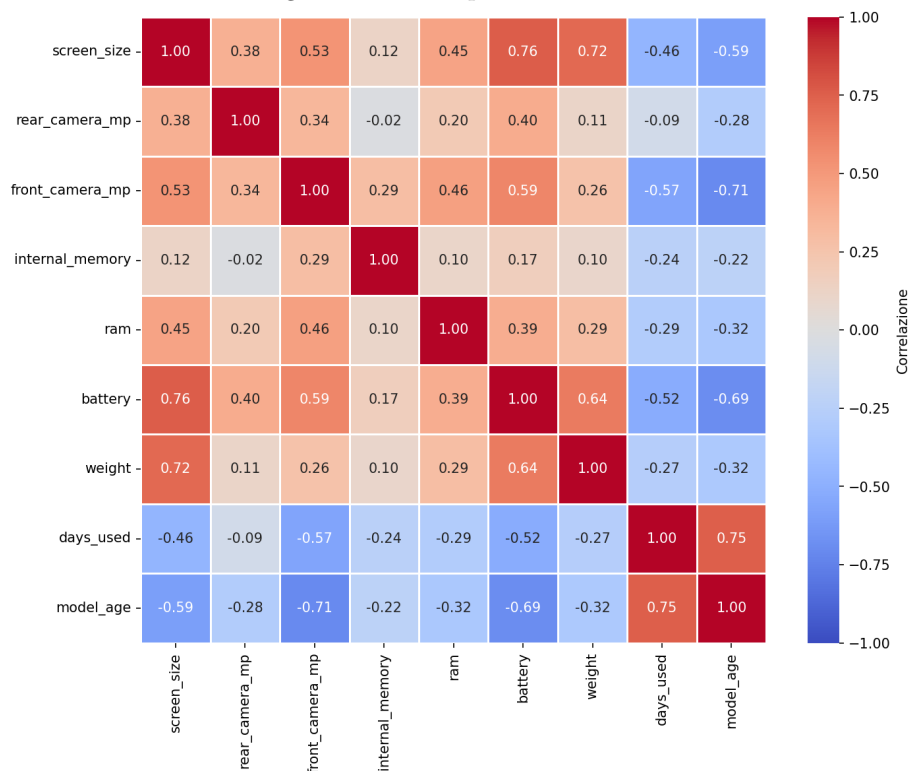
Figura 7: Confronto Matrici Di Confusione



Interpretazione Le fasce **Budget** e **Premium** sono le più facili da classificare (gli estremi della distribuzione) a differenza delle fasce **Mid-Range** e **High-End** dove vi sono più errori, i loro confini sono più sfumati. La **Logistic Regression** ha la diagonale più marcata.

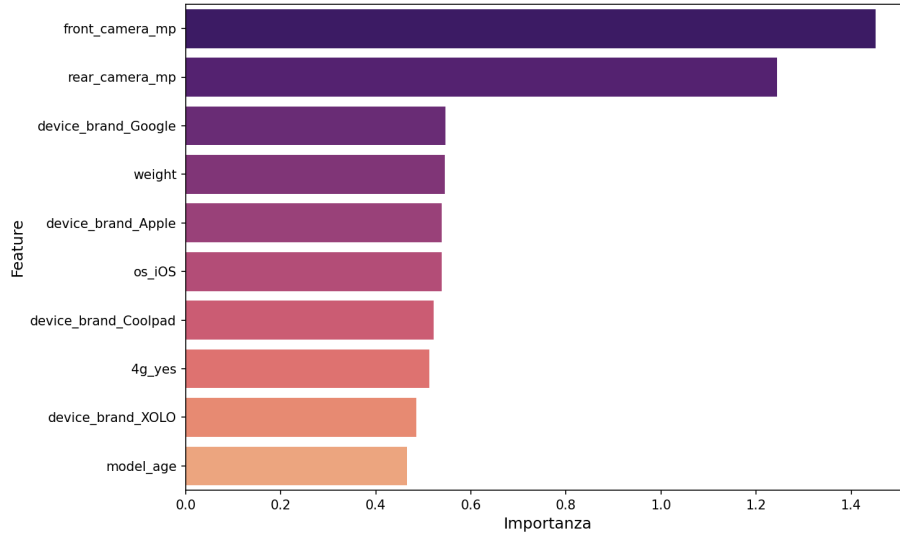
4.4 Correlazione fra le feature

Figura 8: Heatmap di correlazione



Interpretazione In questa matrice possiamo capire quanto le feature fra loro siano correlate, con un indice di correlazione, più alto è l'indice (max 1) più le feature fra loro hanno una forte correlazione. Possiamo notare ad esempio che la batteria (battery) è fortemente correlata alle dimensioni dello schermo (screen_size), questo è di facile intuizione: schermo più grande $\rightarrow$ più spazio $\rightarrow$ batteria più grande. Lo stesso vale per i giorni di utilizzo del telefono (days_used) e la sua età (model_age derivato dall'anno di rilascio del dispositivo), più è vecchio il dispositivo, più giorni di utilizzo avrà.

Figura 9: Feature Importance



Interpretazione Dal grafico delle feature possiamo notare che quelle più importanti (che influiscono positivamente sul prezzo) sono nettamente `front_camera_mp` e `rear_camera_mp`, in quanto ci dicono molto di più sul prezzo di uno smartphone, uno smartphone con un ottimo comparto fotografico costa molto di più. La feature `model_age` invece è quella meno importante in questo grafico (influisce negativamente sul prezzo) , questo perchè chiaramente un dispositivo più vecchio costa sicuramente di meno.

4.5 Motivazione Scelta finale: Logistic Regression

Nonostante le prestazioni dei vari modelli siano abbastanza simili e non c'è nessun modello significativamente più performante, la scelta finale ricade sull'utilizzo del modello **Logistic Regression**, vediamo perchè:

Criterio	LR	RF/GB/RF Tuned
Accuracy	Migliore (65%)	Inferiore (62-64%)
Interpretabilità	Coefficienti diretti	Black Box
Tempo Di Training	Qualche secondo	Da qualche secondo a qualche minuto
Generalizzazione	No overfitting	Rischio overfitting

Tabella 8: Motivazione della scelta finale.

5 Considerazioni finali

5.1 Risultati Raggiunti

1. **Pipeline ML Completa** Dal dato grezzo alla predizione tutto il codice è ben modularizzato ed eseguibile.
2. **PreProcessing robusto** Gestione dei missing values mascherati, degli outlier ed encoding.
3. **Confronto Sistemático** Confronto sistematico con metriche di valutazioni e percentuali fra 4 modelli.
4. **Documentazione Critica** Limitazioni del modello e del dataset ben specificate assieme all'intero processo di sviluppo.

5.2 Limitazioni Riconosciute del modello

Limitazione	Impatto	Possibile Mitigazione
Accuracy del 65%	Non ottima	Aggiungere più feature nel dataset
Data Drift	predizioni errate su entry del 2021+	Retraining periodico su dati aggiornati
No Feature Esterne	Non vengono valutate condizioni fisiche del telefono usato	Integrazione di API Esterne

Tabella 9: Limitazioni Riconosciute.

5.3 Conclusione

Il progetto DealHunter dimostra come affrontare un problema ML reale partendo da un dataset non pulito. L'accuracy del 65% riflette le limitazioni intrinseche dei dati, non della metodologia. Il valore del progetto risiede nella correttezza del processo: preprocessing motivato, valutazione rigorosa, documentazione critica delle scelte e dei limiti.